Name: Sandesh Nakade

Batch: 02

Roll No: 34

PRN No: 12412601

Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Perform following tasks: 1. Pre-process the dataset. 2. Identify outliers. 3. Check the correlation. 4. Implement linear regression and ridge, Lasso regression models. 5. Evaluate the models and compare their respective scores like R2, RMSE, etc. Dataset link: https://www.kaggle.com/datasets/yasserh/uber-fares-dataset

1. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error
```

2. Load Dataset

```
df = pd.read_csv("uber.csv")
df.head()
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_la
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	40.

Step 1: Drop Unwanted Columns

```
df = df.drop(['Unnamed: 0', 'key'], axis=1)
```

```
df.head()
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1.0
1	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1.0
2	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1.0

Step 2: Handle Missing Values

```
df.dropna(inplace=True)
```

Step 3: Datetime Feature Engineering

```
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])

df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month

df.drop('pickup_datetime', axis=1, inplace=True)
```

```
df.head()
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1.0	19	7	5
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1.0	20	17	7
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1.0	21	24	8
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3.0	8	26	6
4	16.0	-73.925023	40.744085	-73.973082	40.761247	5.0	17	28	8

Step 4: Remove Invalid Fares

```
df = df[df['fare_amount'] > 0]
```

```
df
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1.0	19	7	5
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1.0	20	17	7
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1.0	21	24	8
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3.0	8	26	6
4	16.0	-73.925023	40.744085	-73.973082	40.761247	5.0	17	28	8
...	...	...	...	...	...	...	...	...	...
8912	15.0	-73.983202	40.767807	-73.958845	40.809792	1.0	1	16	9
8913	5.5	-73.995007	40.745027	-74.004968	40.746732	2.0	13	9	3
8914	17.3	-73.994098	40.735441	-73.979597	40.689609	1.0	20	1	4
8915	5.5	-73.963379	40.758320	-73.965309	40.760189	1.0	17	16	6
8916	6.1	-73.991367	40.749675	-74.008058	40.730913	5.0	7	21	1

8917 rows × 9 columns

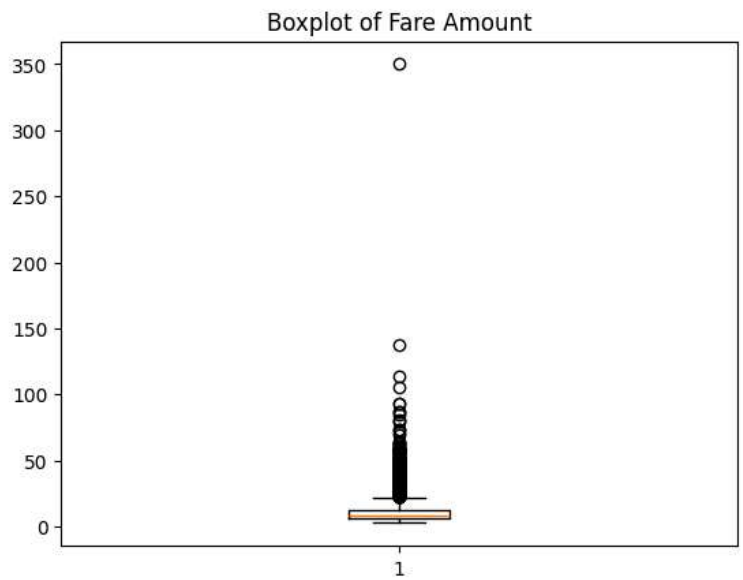
2.Outlier Identification

Fare Amount Outliers

```
Start coding or generate with AI.
```

a. Boxplot for Fare Amount

```
plt.boxplot(df['fare_amount'])
plt.title('Boxplot of Fare Amount')
plt.show()
```



b. Remove Outliers using IQR

```
Q1 = df['fare_amount'].quantile(0.25)
Q3 = df['fare_amount'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['fare_amount'] >= Q1 - 1.5*IQR) &
        (df['fare_amount'] <= Q3 + 1.5*IQR)]
```

3. Feature Engineering- Distance Calculation

```
df = df[(df['fare_amount'] >= Q1 - 1.5*IQR) &
        (df['fare_amount'] <= Q3 + 1.5*IQR)].copy()

df["distance_km"] = haversine(
    df["pickup_latitude"],
    df["pickup_longitude"],
    df["dropoff_latitude"],
    df["dropoff_longitude"]
)
```

3. Correlation Analysis

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

Show hidden output

Observation:

Fare amount is correlated with pickup & drop-off locations

Passenger count has weak correlation

Time features have minor influence

6. Feature Selection And Scaling

```
X = df.drop('fare_amount', axis=1)
y = df['fare_amount']
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Standardization (Important for Ridge & Lasso)

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

7. Model Implementation

a. linear regression

```
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
```

b. ridge regression

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

y_pred_ridge = ridge.predict(X_test)
```

c. Lasso Regression

```
lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)

y_pred_lasso = lasso.predict(X_test)
```

8. Model Evaluation

```
def evaluate_model(y_test, y_pred):
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    return r2, rmse
```

Score

```
lr_r2, lr_rmse = evaluate_model(y_test, y_pred_lr)
ridge_r2, ridge_rmse = evaluate_model(y_test, y_pred_ridge)
lasso_r2, lasso_rmse = evaluate_model(y_test, y_pred_lasso)
```

9. Model Comparison

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression"],
    "R2 Score": [lr_r2, ridge_r2, lasso_r2],
    "RMSE": [lr_rmse, ridge_rmse, lasso_rmse]
})

results
```

	Model	R2 Score	RMSE
0	Linear Regression	-0.005737	4.273053
1	Ridge Regression	-0.005729	4.273034
2	Lasso Regression	-0.005563	4.272682